

DMATH-CODE

Randomisierte Algorithmen

Frühlingssemester 25

Serie 5

Lernziele

- Sie können die erwartete Berechnungskomplexität einfacher Algorithmen berechnen.
- Sie kennen den Unterschied zwischen Las Vegas und Monte Carlo Algorithmen und kennen ihre Vor- und Nachteile.
- Sie können den Code randomisierter Algorithmen korrekt lesen und interpretieren.

Randomisierte Algorithmen nutzen effektiv *Zufälligkeit* zur Optimierung von *Laufzeit* und *Effizienz*. Sie sind in vielen Bereichen wie Kryptographie, maschinelles Lernen, Simulationen und Problemlösen auf Graphen essenziell und sind ein wichtiges Teilgebiet aktueller Forschung.

Für komplexe Probleme sind in vielen Fällen Algorithmen mit Zufallsentscheidungen effizienter und einfacher zu implementieren als deterministische Algorithmen.

Frage 1: Was ist die erwartete Berechnungskomplexität?

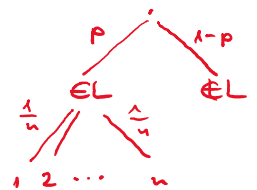
Die **erwartete Berechnungskomplexität** eines Algorithmus gibt an, wie lange ein Algorithmus im Durchschnitt benötigt, um ein Problem mit einer Eingabegrösse n zu lösen. Wir können die Input-Werte Ω des Algorithmus zufällig wählen (zum Beispiel uniform zufällig). Der Erwartungswert $E(X)$ der Zufallsvariable

$$X: \omega \rightarrow \text{Anzahl Rechenschritte bei Eingabe } \omega$$

beschreibt dann die durchschnittliche Laufzeit des Algorithmus.

Aufgabe 1: Bestimmen Sie die **erwartete Berechnungskomplexität** für den `linearSearch()` Algorithmus, wenn das Element mit Wahrscheinlichkeit p in der Liste enthalten ist:

```
def linearSearch(Liste, Element):
    for i in range(len(Liste)):
        if Liste[i] == Element: return True
    return False
```



Input $L = [a_1, a_2, a_3, \dots, a_n]$; $P(\text{El} \in L) = p$

x	3	5	7	9	...	$2n+1$	$2n+2$
$P(X=x)$	$p \cdot \frac{1}{n}$	$p \cdot \frac{1}{n}$	$p \cdot \frac{1}{n}$	$p \cdot \frac{1}{n}$	...	$p \cdot \frac{1}{n}$	$1-p$

$$\text{Test: } \underbrace{\frac{p}{n} + \frac{p}{n} + \frac{p}{n} + \frac{p}{n} + \dots + \frac{p}{n}}_{= n \cdot \frac{p}{n} = p} + 1-p = p + 1-p = \underline{\underline{1}}$$

$$\begin{aligned} E(X) &= 3 \cdot \frac{p}{n} + 5 \cdot \frac{p}{n} + 7 \cdot \frac{p}{n} + \dots + (2n+1) \cdot \frac{p}{n} + (2n+2) \cdot (1-p) \\ &= \frac{p}{n} \cdot \left(\underbrace{1 + 3 + 5 + 7 + \dots + (2n+1)}_{\substack{k=1 \quad k=2 \quad k=3 \quad k=n}} - 1 \right) + (2n+2) \cdot (1-p) \end{aligned}$$

$$\begin{aligned} &= \frac{p}{n} \cdot \left((n+1)^2 - 1 \right) + (1-p) \cdot (2n+2) \\ &= \frac{p}{n} \cdot (n^2 + 2n + 1 - 1) + 2n + 2 - 2np - 2p \\ &= \frac{p \cdot n}{n} + \frac{2p}{n} + 2n + 2 - 2np - 2p \\ &= 2n - p \cdot n + 2 \\ &= \underline{\underline{(2-p) \cdot n + 2}} \in \Theta(n^1) \end{aligned}$$

$$\begin{aligned} k & \\ 1 &: 1+3=4=2^2 \\ 2 &: 1+3+5=9=3^2 \\ 3 &: 1+3+5+7=16=4^2 \\ &\vdots \\ n &: 1+3+5+\dots+(2n+1)=(n+1)^2 \end{aligned}$$

Aufgabe 2: Bestimmen Sie die *erwartete Berechnungskomplexität* für den `insertionSort()` Algorithmus für eine Liste der Länge 5 mit paarweise verschiedenen Elementen.

Optional: Bestimmen Sie eine allgemeine Formel für Listen der Länge n .

```
def insertionSort(Liste):
    for i in range(1, len(Liste)):
        j = i
        while j > 0 and Liste[j] < Liste[j-1]:
            temp = Liste[j]
            Liste[j] = Liste[j-1]
            Liste[j-1] = temp
            j = j-1
```

X_i = Anzahl Rechenschritte im while-loop

x	2	8
$P(X_1 = x)$	$1/2$	$1/2$

$$E(X_1) = \frac{1}{2} \cdot 2 + \frac{1}{2} \cdot 8 = 5$$

1 2	2 1	3 1	4 1	5 1
1 3	2 3	3 2	4 2	5 2
1 4	2 4	3 4	4 3	5 3
1 5	2 5	3 5	4 5	5 4

x	2	8	14
$P(X_2 = x)$	$1/3$	$1/3$	$1/3$

$$E(X_2) = \frac{1}{3} \cdot 2 + \frac{1}{3} \cdot 8 + \frac{1}{3} \cdot 14 = 8$$

1 2 3	1 3 2	1 4 2	1 5 2	2 3 1	2 4 1	2 5 1	3 4 1	3 5 1	4 5 1
1 2 4	1 3 4	1 4 3	1 5 3	2 3 4	2 4 3	2 5 3	3 4 2	3 5 2	4 5 2
1 2 5	1 3 5	1 4 5	1 5 4	2 3 5	2 4 5	2 5 4	3 4 5	3 5 4	4 5 3

x	2	8	14	20
$P(X_3 = x)$	$1/4$	$1/4$	$1/4$	$1/4$

$$E(X_3) = \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 8 + \frac{1}{4} \cdot 14 + \frac{1}{4} \cdot 20 = 11$$

1 2 3 4	1 2 4 3	1 2 5 3	1 3 4 2	1 3 5 2	1 4 5 2	2 3 4 1	2 3 5 1	2 4 5 1	3 4 5 1
1 2 3 5	1 2 4 5	1 2 5 4	1 3 4 5	1 3 5 4	1 4 5 3	2 3 4 5	2 3 5 4	2 4 5 3	3 4 5 2

x	2	8	14	20	26
$P(X_4 = x)$	$1/5$	$1/5$	$1/5$	$1/5$	$1/5$

$$E(X_4) = \frac{1}{5} \cdot 2 + \frac{1}{5} \cdot 8 + \frac{1}{5} \cdot 14 + \frac{1}{5} \cdot 20 + \frac{1}{5} \cdot 26 = 14$$

1 2 3 4 5	1 2 3 5 4	1 2 4 5 3	1 3 4 5 2	2 3 4 5 1
-----------	-----------	-----------	-----------	-----------

$$X = 2 + X_1 + 2 + X_2 + 2 + X_3 + 2 + X_4$$

$$\begin{aligned} E(X) &= 2 + E(X_1) + 2 + E(X_2) + 2 + E(X_3) + 2 + E(X_4) \\ &= 2 + 5 + 2 + 8 + 2 + 11 + 2 + 14 = \underline{\underline{46}} \end{aligned}$$

Frage 2: Was sind randomisierte Algorithmen?

Ein **randomisierter Algorithmus** ist ein Algorithmus, der Zufallszahlen oder Zufallsentscheidungen während seiner Berechnungen verwendet. Dadurch kann sich seine **Laufzeit** oder das **Ergebnis** bei jeder Ausführung unterscheiden. Aber warum sind zufällige Entscheidungen in einer algorithmischen Berechnung hilfreich?

- Um **schwierige Probleme** effizienter zu lösen.
- Um **Worst-Case-Szenarien** zu vermeiden.
- Um **Approximationen** mit Zufall zu berechnen.

Man unterscheidet **zwei Hauptkategorien** von randomisierten Algorithmen: Las Vegas-Algorithmen und Monte Carlo-Algorithmen, die sich hinsichtlich ihrer **Korrektheit** und ihrem **Laufzeitverhalten** unterscheiden.

- Ein **Las Vegas-Algorithmus** gibt immer ein korrektes Ergebnis zurück, nutzt jedoch zufällige Entscheidungen, wodurch seine Laufzeit variieren kann. Er beendet die Berechnung erst, wenn eine gültige Lösung gefunden wurde. Das bedeutet, dass die Anzahl der benötigten Schritte nicht fest ist, aber die Lösung stets richtig ist.
- Ein **Monte Carlo-Algorithmus** gibt das Ergebnis innerhalb einer festgelegten Laufzeit zurück, es kann aber mit einer bestimmten Wahrscheinlichkeit falsch sein. Diese Fehlerwahrscheinlichkeit kann jedoch oft durch mehrfaches Ausführen beliebig reduziert werden.

Aufgabe 3: Zu welcher Kategorie gehört folgender Algorithmus:

```
def randQuicksort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = rd.choice(arr) zufällig  
    L = [x for x in arr if x < pivot]  
    M = [x for x in arr if x == pivot]  
    R = [x for x in arr if x > pivot]  
    return randQuicksort(L) + M + randQuicksort(R)
```

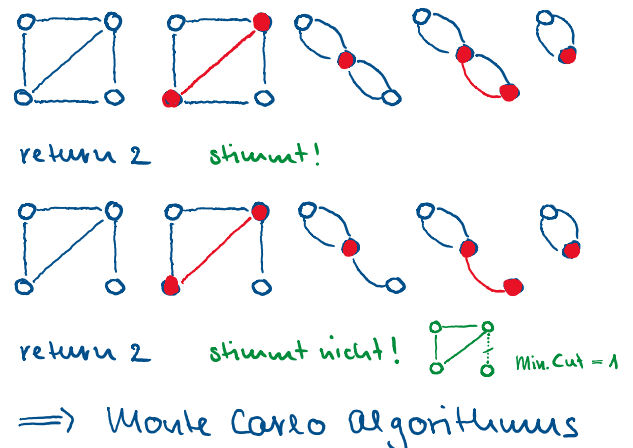
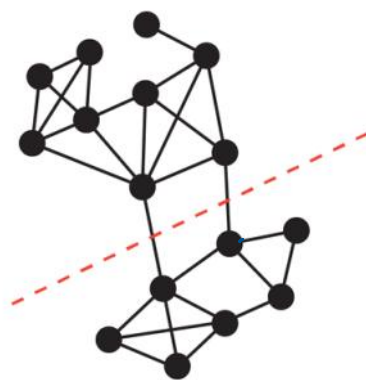
Aufteilung abhängig von pivot (bezieht sich auf L, M, R)
korrekte Zusammensetzung (bezieht sich auf die Rekursion)

Der Algorithmus sortiert immer korrekt. Nur die Aufteilung in L, M und R ist zufällig. $\Rightarrow$ Las Vegas-Algorithmus

Aufgabe 3: Gegeben ist ein ungerichteter Graph $G = (V, E)$, also eine Menge von Knoten V und Kanten E . Ein **Minimum Cut** ist eine kleinste Menge von Kanten, deren Entfernung den Graphen in zwei getrennte Komponenten teilt.

Zu welcher Kategorie gehört der folgende Algorithmus für das Minimum Cut Problem:

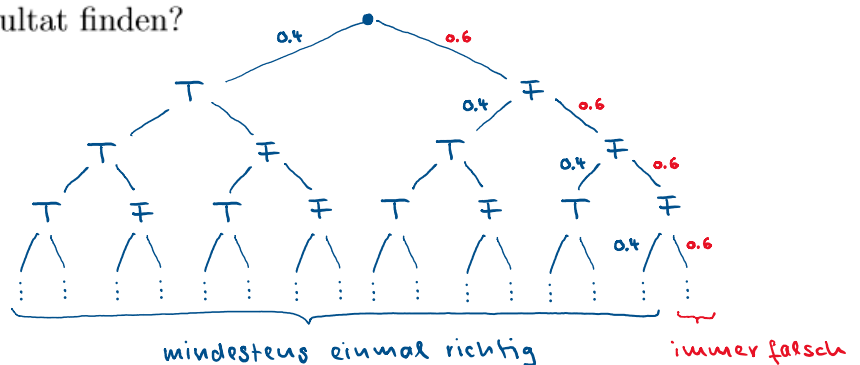
1. Wähle uniform zufällig eine Kante $\{U, V\}$.
2. Verschmelze die beiden Knoten U und V zu einem neuen Knoten und entferne alle Kanten zwischen diesen beiden Knoten.
3. Wiederhole das, bis nur noch zwei Knoten übrig sind.
4. Gib die Anzahl der verbleibenden Kanten zurück.



Aufgabe 4: Wenn die Wahrscheinlichkeit für eine korrekte Lösung bei einem Monte Carlo-Algorithmus 40% beträgt, wie oft müssen Sie dann den Algorithmus durchführen, damit Sie mit einer Wahrscheinlichkeit von 99.9% ein korrektes Resultat finden?

T = Lösung richtig
F = Lösung falsch

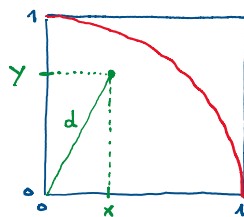
$$\begin{aligned}
 &P(\text{mind. 1 mal richtig}) \\
 &= 1 - P(\text{immer falsch}) \\
 &= 1 - 0.6^n = 0.999 \\
 \Leftrightarrow &1 - 0.999 = 0.6^n \\
 \Leftrightarrow &0.001 = 0.6^n \\
 \Leftrightarrow &n = \log_{0.6}(0.001) \\
 &\approx 13.522...
 \end{aligned}$$



Wenn wir also 14 Lösungen berechnen, ist mit Wahrscheinlichkeit 99.9% mind. eine korrekt.

Aufgabe 5: Was berechnet der folgende Algorithmus und zu welcher Kategorie gehört er?

```
def monteCarloEstimator(n):
    x = np.random.uniform(0,1,n)
    y = np.random.uniform(0,1,n)
    test = [1 for i in range(n) if x[i]**2 + y[i]**2 <= 1]
    return 4 * sum(test)/n
```



Anteil der Punkte,
die im Viertelkreis
mit Radius 1 liegen
 $\approx \frac{1}{4} \cdot 1^2 \cdot \pi$
 $= \frac{\pi}{4}$

Für jeden zufällig erzeugten Punkt, der
im Kreis $x^2 + y^2 \leq 1^2$ liegt, wird in «test»
eine 1 gespeichert.

Dieser Monte-Carlo Algorithmus approximiert π

Aufgabe 6: Was berechnet der folgende Algorithmus und zu welcher Kategorie gehört er?

Optional: Zeigen Sie, dass die erwartete Berechnungskomplexität zu $O(n)$ gehört!

Gibt das i -te Element der sortierten Liste A zurück.

```
def randSelect(A,i):
    n = len(A)
    if n==1:
        return A[n-1]
    pivot = np.random.randint(n)
    L, R = [], []
    for j in range(len(A)):
        if j != pivot:
            if A[j] < A[pivot]:
                L.append(A[j])
            else:
                R.append(A[j])
    if i < len(L):
        return randSelect(L,i)
    elif i == len(L):
        return A[pivot]
    else:
        return randSelect(R,i-(len(L)+1))
```

zufällige Aufteilung in
 $[\dots] < x \leq [\dots]$
L A[pivot] R

- |L| Elemente sind kleiner als x.
- |A|-|L|-1 Elemente sind grösser gleich als x.

das i -te Element in der
sortierten Liste L suchen
x ist das i -te Element in
der sortierten Liste A.

das $(i-|L|-1)$ -te
Element in der
sortierten Liste R
suchen.

immer korrekt
 $\Rightarrow$ Las Vegas-Algorithmus

$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ \text{wäre} & [1, 1, 2, 4, 5, 7, 7, 8, 8, 8] \end{matrix}$

z.B.: $A = [7, 5, 1, 8, 8, 7, 4, 1, 2, 8]; i = 6$

zufällig: pivot = 6 $\begin{matrix} 0 & 1 & 2 \\ \text{wäre} & [5, 7, 7, 8, 8, 8] \end{matrix}$

$[1, 1, 2] \quad 4 \quad [7, 5, 8, 8, 7, 8]$
 $\leftarrow 3 \rightarrow \quad \leftarrow 6 \rightarrow$

$i > 3 \Rightarrow$ rechts das $i = (6 - 3 - 1) = 2$ -te Element suchen

zufällig: pivot = 3

$\begin{matrix} 0 & 1 & 2 \\ \text{wäre} & [5, 7, 7] \end{matrix}$

$[7, 5, 7] \quad 8 \quad [8, 8]$
 $\leftarrow 3 \rightarrow \quad \leftarrow 2 \rightarrow$

$i < 3 \Rightarrow$ links das $i = 2$ -te Element suchen

zufällig: pivot = 0

$\begin{matrix} 0 \\ \text{wäre} & [7] \end{matrix}$

$[5] \quad 7 \quad [7]$
 $\leftarrow 1 \rightarrow \quad \leftarrow 1 \rightarrow$

$i > 1 \Rightarrow$ rechts das $i = (2 - 1 - 1) = 0$ -te Element suchen

$|[7]| = 1: \text{ return } \underline{\underline{7}}$